

SIH26131

FULL SOFTWARE BLUEPRINT

Software Requirements • Digital Hardware Simulator • AI Training • APIs • Database • Testing
• Roadmap

Engineering master document for building the complete software prototype without purchasing physical hardware.

Design principle: build the production architecture first; use simulated weather, soil, leaf-wetness and pest-trap telemetry during the hackathon; preserve the same interfaces for future real devices.

Area	Definition
Product	Crop Health Intelligence Platform
Primary users	Farmers, experts/extension workers, agriculture officers
Core intelligence	Image AI + severity + environment + pest + history + GIS
Hardware strategy	Digital simulator now; physical device adapters later
Primary output	Detection, risk, hotspot, validated advisory, alert, follow-up
Development style	Modular services, versioned APIs, measurable acceptance criteria

Version 1.0 • 27 August 2026

1. Purpose & Scope

This document converts the project concept into an implementation blueprint. It defines what must be built, how modules communicate, how AI is trained, how hardware is simulated, what data is stored, how each goal is verified, and what constitutes completion.

The project is intentionally scoped as a software solution. No physical IoT hardware is required for the hackathon demonstration. The architecture must nevertheless be ready to accept future real-device telemetry.

2. Product Objectives

Objective	Required outcome
O1 — Detect	Identify supported crop diseases/pests from images.
O2 — Quantify	Estimate severity using segmentation or validated severity classification.
O3 — Contextualize	Combine environmental, pest, crop-stage and historical evidence.
O4 — Predict	Detect rising risk and generate early warnings.
O5 — Locate	Map cases/risk to farm zones and identify surveillance hotspots.
O6 — Validate	Allow experts to confirm/correct uncertain AI results.
O7 — Act	Provide approved, contextual and multilingual management guidance.
O8 — Monitor	Create alerts, rechecks and outcome history.
O9 — Learn	Store feedback and validated labels for future evaluation/training.
O10 — Integrate	Use one telemetry contract for simulator and future real hardware.

3. Non-Goals

- Universal disease diagnosis across every crop.
- Unverified chemical prescriptions or dosage generation.
- Pretending simulated telemetry is real sensor data.
- Automatic online model retraining unless a real retraining pipeline is implemented.
- Replacing agronomists or laboratory confirmation in high-uncertainty cases.

4. User Roles & Permissions

Role	Permissions
Farmer	Manage own farm, scan, view results, receive alerts, provide feedback, request review.
Expert	Review assigned cases, confirm/correct diagnosis, annotate, comment.
Officer	View regional/farm analytics, hotspots, trends and validated cases.
Admin	Manage users, crops, classes, devices, knowledge base, model deployments.
ML/Developer	Model evaluation, dataset/version metadata, simulator controls in development environment.

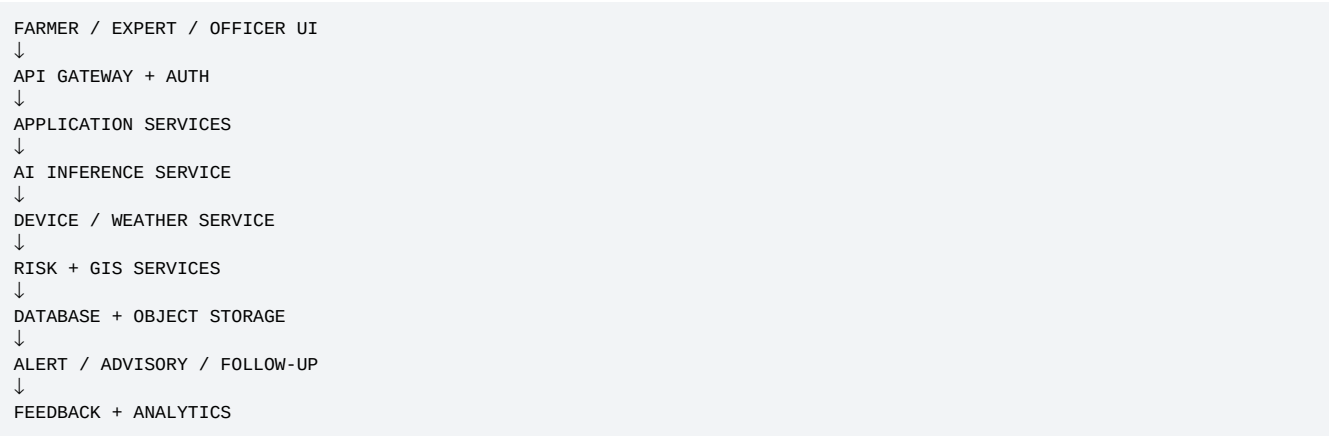
5. Functional Requirements

ID	Requirement	Priority
FR-01	User can register/select farm, crop and zone.	Must
FR-02	User can capture/upload crop image.	Must
FR-03	System validates image quality before inference.	Must
FR-04	System returns supported disease/pest prediction with confidence.	Must
FR-05	System stores model/version metadata with prediction.	Must
FR-06	System calculates severity where supported.	Should
FR-07	System accepts telemetry from simulator through API.	Must
FR-08	System calculates zone-level risk.	Must
FR-09	System detects rapid risk increase.	Should
FR-10	System displays GIS/zone hotspot view.	Must
FR-11	Expert can confirm/correct prediction.	Must
FR-12	System selects approved advisory content.	Must
FR-13	System supports English/Tamil core UX.	Should
FR-14	System creates and tracks alerts.	Must
FR-15	System schedules/stores follow-up observations.	Should
FR-16	System stores feedback/outcomes.	Must
FR-17	Future real device can use same telemetry contract.	Must

6. Non-Functional Requirements

- Reliability: the complete demo must work without physical hardware or external weather dependency.
- Performance: inference and dashboard updates must be measured and optimized for the chosen deployment.
- Security: authentication, authorization, file validation, secrets management and rate limiting.
- Explainability: show confidence, evidence and data freshness; do not present risk as certainty.
- Maintainability: modular services, versioned APIs and documented schemas.
- Traceability: store model version, advisory version and important validation actions.

7. System Architecture



8. End-to-End Master Flow



9. Service Boundaries

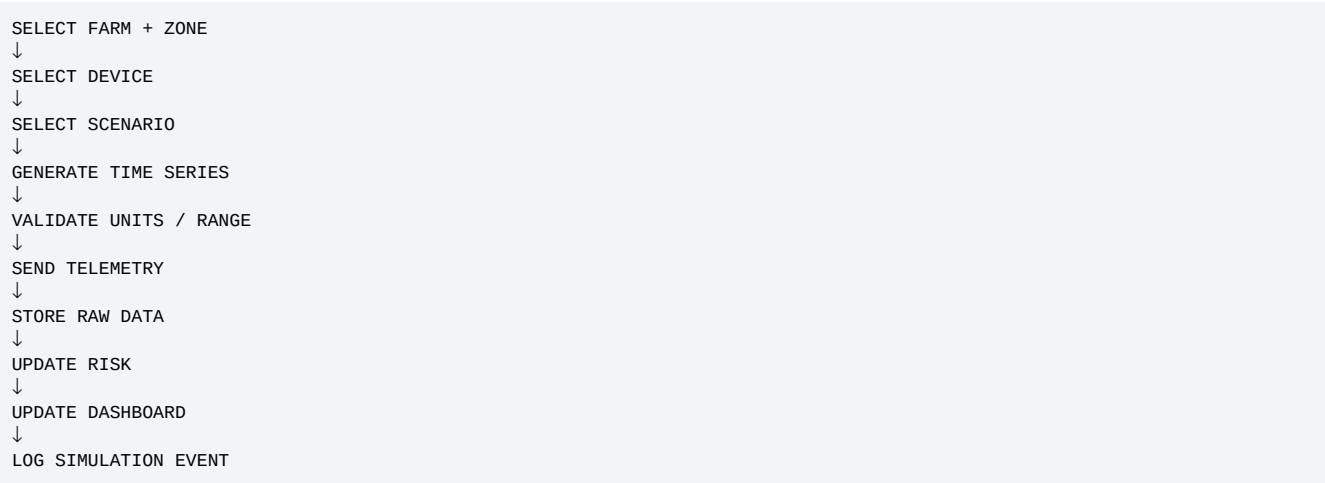
Service	Owns
Auth	Users, sessions, roles, permissions
Farm	Farms, zones, crops, crop stage
Scan	Image lifecycle and scan state
AI	Model inference and prediction objects
Device	Telemetry ingestion, device health
Feature	Time windows and derived variables
Risk	Score, band, trend, reasons
GIS	Zone geometry and hotspot aggregation
Expert	Review queue and validation
Advisory	Approved knowledge selection
Alert	Event detection and alert state
Feedback	Corrections, outcomes, learning dataset metadata

10. Digital Hardware Simulator — Requirements

The simulator is a first-class development module. It must reproduce realistic device behavior, send standard telemetry, support repeatable scenarios and deliberately produce failures for testing.

Virtual device	Telemetry	Controls
Weather node	Temperature, humidity, rainfall, wind	Baseline/trend/rain
Soil node	Soil moisture, soil temperature	Dry/wet trend
Leaf-wetness node	Wetness level/duration	Rain/dew
Pest trap	Pest type, count, timestamp	Stable/rising/spike
Gateway	Connectivity, last seen, health	Offline/reconnect

11. Simulator Flow



12. Required Simulator Scenarios

Scenario	Expected behavior
Normal day	Stable readings; low/normal risk.
Rain event	Rainfall/humidity/leaf wetness rise; risk features respond.
Humid night	Humidity and wetness remain elevated; risk trend may rise.
Pest spike	Pest counts increase rapidly; pest-risk feature increases.
Disease-risk event	Multiple contextual variables become favorable; risk rises.
Sensor offline	Device becomes unavailable; stale-data warning appears.
Bad telemetry	Out-of-range/invalid values rejected and logged.
Recovery	Device reconnects; readings resume and status recovers.

13. Repeatability Rule

Every judge-facing scenario must have a fixed seed or deterministic event sequence so the team can reproduce the same demonstration. The simulator should support reset and replay.

14. Telemetry Contract

```
{ "device_id": "SIM-WEATHER-001", "zone_id": "ZONE-B", "timestamp": "ISO-8601", "temperature_c": 28.4, "humidity_pct": 84, "soil_moisture_pct": 61, "leaf_wetness": 0.72, "pest_count": 12, "data_source": "simulated" }
```

Future device adapters should publish equivalent logical fields. Add `firmware_version`, calibration metadata, battery/connectivity state and sensor-health flags when physical hardware exists.

15. Telemetry Validation

- Validate required IDs and timestamp.
- Validate numeric ranges and units.
- Reject impossible values.
- Record data source and device status.
- Handle duplicate/out-of-order messages.
- Record ingestion timestamp separately from measurement timestamp.
- Mark stale data when the device stops reporting.

16. Weather Adapter

Implement an abstraction with two providers: `SimulatorProvider` for the hackathon and `ExternalWeatherProvider` for later integration. Do not hard-code the risk engine to one provider.

17. Device State Machine

```
ONLINE
↓
NO MESSAGE FOR THRESHOLD
↓
STALE
↓
CONTINUED ABSENCE
↓
OFFLINE
↓
NEW VALID MESSAGE
↓
ONLINE
↓
INVALID MESSAGE
↓
QUARANTINE / LOG
```

18. AI Training Plan — Goal by Goal

AI work must be treated as an engineering pipeline, not only model selection.

Goal A — Disease classification

- 1 Choose the primary crop and exact supported classes.
- 2 Collect/obtain a legally usable dataset.
- 3 Inspect class balance and image quality.
- 4 Remove duplicates and prevent same-plant leakage.
- 5 Create train/validation/test splits.
- 6 Train transfer-learning baselines.
- 7 Compare macro F1, per-class recall and latency.
- 8 Select and version the best model.
- 9 Test on field-style images.
- 10 Calibrate confidence thresholds.

Goal B — Pest detection

- 1 Define pest taxonomy.
- 2 Obtain bounding-box annotations.
- 3 Train detector baseline.
- 4 Evaluate precision, recall and mAP.
- 5 Test small/occluded pests.
- 6 Export inference model only if performance is credible.

Goal C — Severity

- 1 Define severity labels or pixel masks.
- 2 Choose segmentation or severity classification.
- 3 Train baseline.
- 4 Calculate affected-area percentage if segmentation is used.
- 5 Compare model output with human labels.
- 6 Set safe bands only after validation.

19. AI Training Flowchart

```
DEFINE CROP + CLASSES
↓
DATASET AUDIT
↓
CLEAN / DEDUPLICATE
↓
ANNOTATE
↓
SPLIT WITHOUT LEAKAGE
↓
AUGMENT TRAINING DATA
↓
TRAIN BASELINE
↓
VALIDATE
↓
FIELD-STYLE TEST
↓
ERROR ANALYSIS
↓
CALIBRATE CONFIDENCE
↓
MODEL VERSION
↓
DEPLOY
↓
MONITOR
```

20. Dataset Requirements

Requirement	Why
Class definitions	Prevents ambiguous labels.
Source metadata	Tracks provenance and dataset version.
Field-style samples	Tests domain shift.
Balanced evaluation	Avoids misleading overall accuracy.
Hard examples	Blur, shadow, occlusion and mixed symptoms.
Separate test set	Prevents tuning against final results.

21. Model Output Contract

```
scan_id model_name model_version crop prediction confidence class_probabilities severity_percent severity_band
pest_detections[] inference_time_ms created_at
```

22. AI Quality Gate

```
IMAGE
↓
QUALITY SCORE
↓
SUPPORTED CROP
↓
MODEL CONFIDENCE
↓
THRESHOLD PASSED?
↓
YES → RESULT
↓
NO → UNCERTAIN / RETAKE / EXPERT
```

23. AI Failure Policy

Low confidence must never be converted into a confident-looking answer. The UI should show uncertainty and route the case to retake or expert review.

24. Model Versioning

For every deployed model store: model name, version, dataset version, training date, metrics, class taxonomy, preprocessing version and deployment status.

25. Environmental Feature Engine

Raw signal	Derived features
Humidity	6/12/24h mean, max, duration above threshold, trend
Temperature	mean, min/max, range, trend
Rainfall	recent total, event flag
Leaf wetness	duration, recent wetness
Soil moisture	current, slope/trend
Pest count	current, moving average, growth rate, spike flag

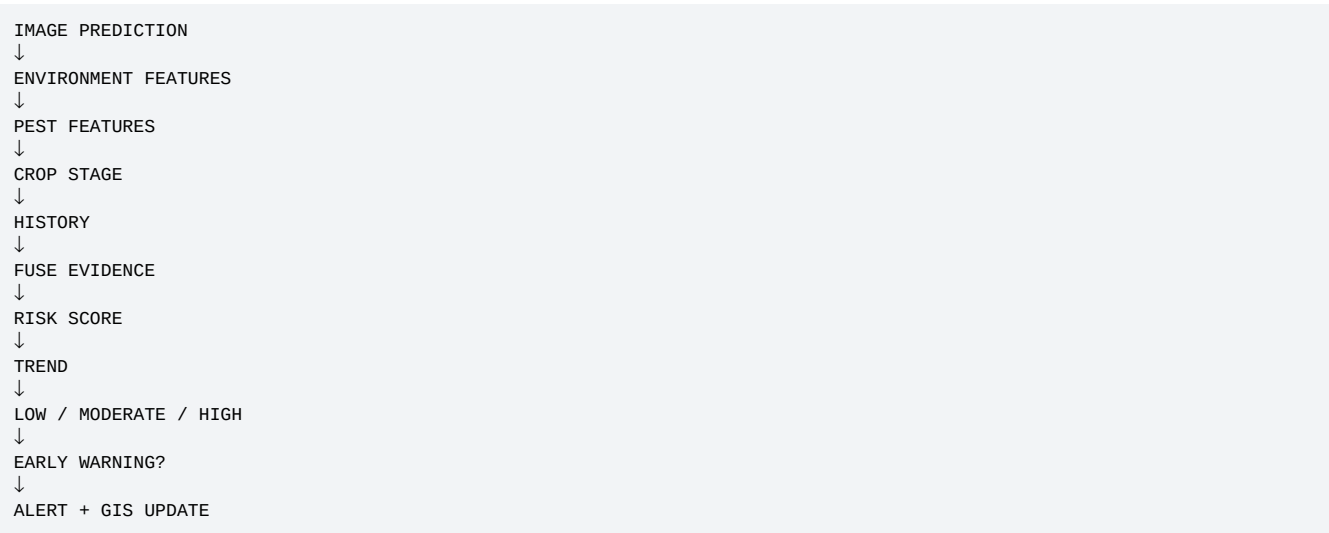
26. Multimodal Risk Engine

Use a transparent prototype score while field data is limited. The design must allow later replacement by a trained/calibrated forecasting model.

```
risk = clamp( w_ai * ai_probability + w_env * environment_risk + w_pest * pest_risk + w_severity * severity + w_history * historical_risk, 0, 1)
```

Return: score 0-100, risk band, trend, evidence/reasons, data freshness and model/rule version.

27. Risk Flowchart



28. Explainability

Show the major contributing signals. Example: “AI confidence high + humidity trend rising + pest count increasing.” Do not state causal certainty unless scientifically validated.

29. GIS / Zone Model

Create a farm polygon or schematic zone grid for the prototype. Every case and device belongs to a zone. Later, real coordinates can be used.

30. Hotspot Flow

```
RECENT CASES
↓
ZONE RISK
↓
SEVERITY TREND
↓
PEST TREND
↓
SPATIAL NEIGHBORHOOD
↓
HOTSPOT SCORE
↓
SURVEILLANCE HOTSPOT
↓
DISPLAY ON MAP
```

31. Expert Validation

```
AI CASE
↓
CONFIDENCE + EVIDENCE
↓
EXPERT QUEUE
↓
CONFIRM / CORRECT / INSUFFICIENT
↓
SAVE ORIGINAL + FINAL LABEL
↓
UPDATE CASE
↓
FEEDBACK DATASET
```

32. Advisory Engine

Create a controlled knowledge base keyed by crop, diagnosis, severity/risk and context. Store source, version and review date. The recommendation service selects approved content; it does not invent agronomic prescriptions.

33. Multilingual Flow

```
STRUCTURED RESULT
↓
APPROVED TEXT
↓
LANGUAGE SELECTOR
↓
ENGLISH / TAMIL
↓
FARMER DISPLAY
```

34. Database Blueprint

Table	Key fields
users	id, role, language, status
farms	id, owner_id, crop, area
zones	id, farm_id, name, geometry
devices	id, zone_id, type, source, status
sensor_readings	device_id, zone_id, timestamp, values, source
scans	id, user_id, zone_id, image_id, status
predictions	scan_id, model_version, class, confidence
severity_results	scan_id, method, area_percent, band
risk_events	zone_id, timestamp, score, band, reasons
expert_reviews	scan_id, reviewer, decision, corrected_label
knowledge_items	crop, diagnosis, severity, action, source, version
alerts	zone_id, type, severity, state, created_at
followups	case_id, due_at, result
feedback	case_id, label, outcome, note
models	name, version, dataset, metrics, status

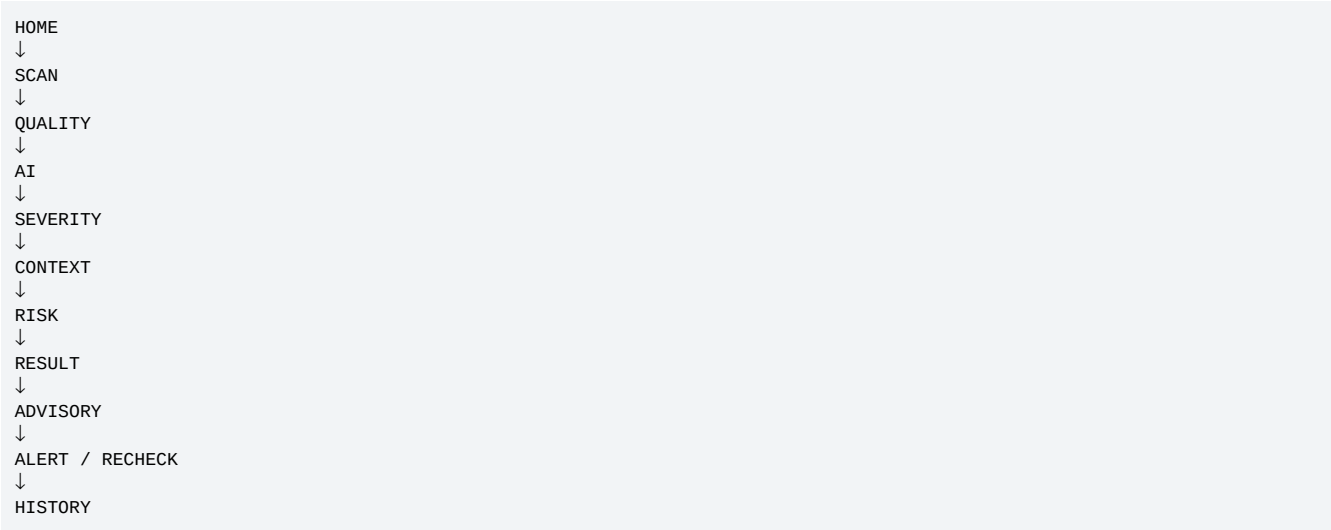
35. API Blueprint

Method / endpoint	Purpose
POST /api/v1/scans	Create scan
POST /api/v1/images/quality	Run quality gate
GET /api/v1/scans/{id}	Get case/result
POST /api/v1/telemetry	Ingest simulator/future device data
GET /api/v1/devices/{id}/latest	Latest telemetry
GET /api/v1/zones/{id}/risk	Current risk
GET /api/v1/farms/{id}/map	GIS layer data
POST /api/v1/expert/reviews	Submit validation
POST /api/v1/feedback	Store correction/outcome
GET /api/v1/alerts	List alerts
POST /api/v1/simulator/scenario	Start/replay scenario
GET /api/v1/models	Model registry metadata

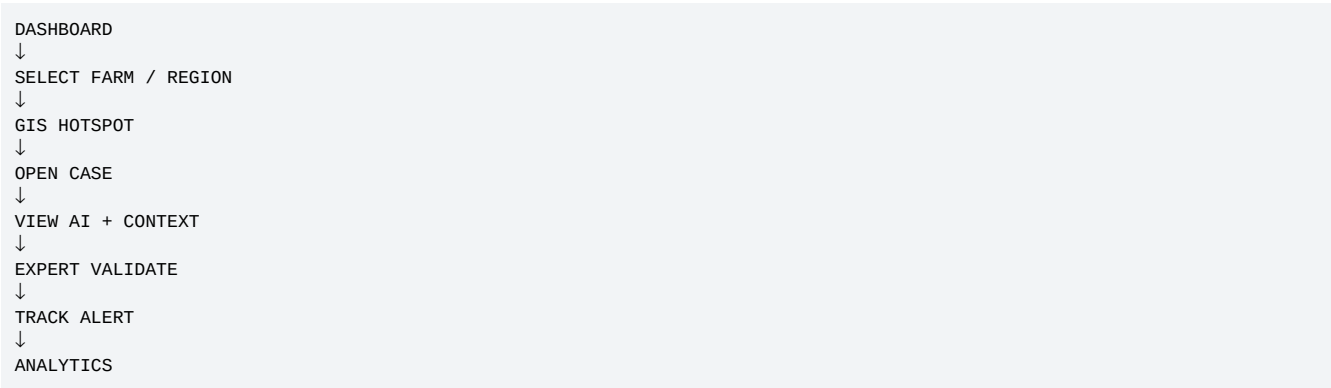
36. UI / Screen Blueprint

Screen	Required content
Farmer Home	Farm health, current risk, latest alert, device data freshness
Scan	Crop/zone + camera/upload + capture guidance
Analysis	Quality + inference state
Result	Diagnosis, confidence, severity, evidence, risk, action
Farm Map	Zone risk, hotspot, cases, device status
History	Scans, risk/severity trends, feedback
Alerts	New/acknowledged/resolved alerts
Expert Queue	Cases with confidence/evidence and review controls
Officer Dashboard	Regional/farm summary, hotspots, trends
Simulator	Devices, telemetry, scenarios, replay, failure controls
Admin	Users, crop classes, knowledge base, models, devices

37. Farmer Result Flow



38. Officer Flow



39. Development Phases — Detailed

Phase	Work packages	Exit criterion
P0	Scope, crop/classes, architecture, contracts	Approved blueprint
P1	Dataset audit, labeling, baseline	Reproducible training run
P2	Backend, auth, DB, object storage	APIs tested
P3	Farmer scan + AI integration	Image→prediction works
P4	Quality + severity	Bad input handled; severity works if included
P5	Simulator + telemetry	Scenarios send/store data
P6	Feature + risk engine	Risk updates from evidence
P7	GIS	Zones/hotspots visible
P8	Expert validation	Confirm/correct workflow works
P9	Advisory + language	Approved advice displayed
P10	Alerts + follow-up	Lifecycle works
P11	Analytics	KPIs/trends available
P12	Security + resilience	Failure tests pass
P13	Demo hardening	Repeatable 3-minute demo
P14	Pitch artifacts	Architecture, metrics, limitations ready

40. Goal-by-Goal Build Order

- 1 G01 disease AI
- 2 G02 image quality
- 3 G03 severity
- 4 G04 digital telemetry
- 5 G05 environmental features
- 6 G06 pest intelligence
- 7 G07 risk/forecast
- 8 G08 GIS hotspots
- 9 G09 expert validation
- 10 G10 advisory
- 11 G11 alerts
- 12 G12 follow-up
- 13 G13 feedback
- 14 G14 multilingual
- 15 G15 hardware integration readiness

41. Test Plan

Test level	Examples	Pass condition
Unit	Risk formula, range checks, severity calculation	Expected values
AI	Held-out and field-style images	Metrics meet declared threshold
API	Auth, upload, telemetry, errors	Correct status/schema
Integration	Simulator→risk; scan→AI→DB	End-to-end state correct
GIS	Zone risk/hotspot	Correct zone state
Expert	Confirm/correct/insufficient	Audit record stored
Resilience	Offline sensor, bad image, AI unavailable	Safe fallback
Security	Unauthorized role, invalid file	Rejected safely
E2E	Complete farmer scenario	Demo story completes

42. Acceptance Criteria

- No physical hardware is required for the core demo.
- Simulator data is clearly labelled simulated.
- Future hardware uses the same telemetry contract.
- A supported image can complete the scan workflow.
- Poor images produce retake/uncertain behavior.
- Risk score includes documented evidence.
- Expert correction is persisted.
- GIS reflects case/risk state.
- Advisory content is controlled and versioned.
- Follow-up changes the case history.
- Model metrics and limitations are visible to the team.
- The demo is reproducible from a clean deployment.

43. Security Checklist

- RBAC implemented.
- Uploads validated and size-limited.
- Secrets outside source code.
- HTTPS for network deployments.
- Device IDs/credentials separated from user accounts.
- Audit logs for expert/admin actions.
- Rate limits on expensive endpoints.
- Minimal personal data storage.

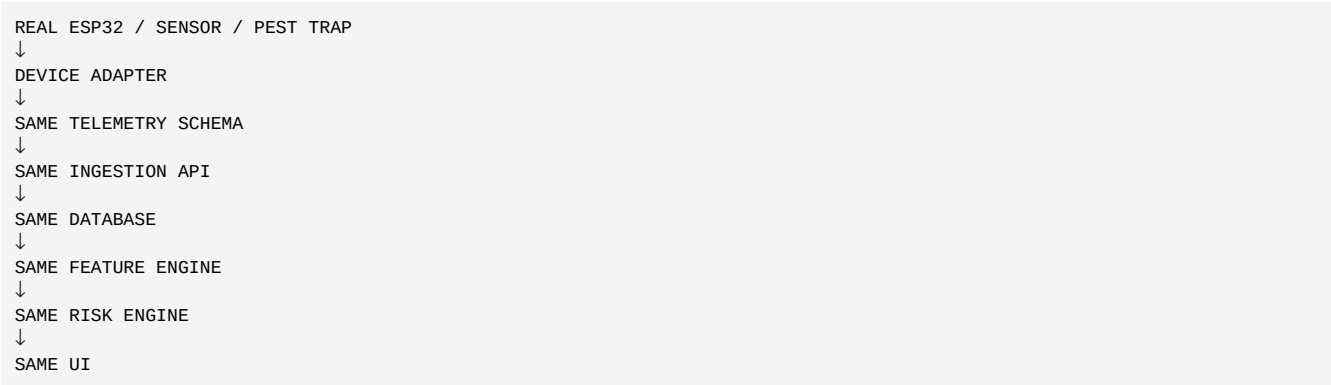
44. Winning Demo Scenario



45. Three-Minute Demo Script

Time	Action
0:00-0:20	State the problem and show farm.
0:20-0:45	Trigger simulator event; show telemetry and rising risk.
0:45-1:15	Scan crop image; show quality and AI.
1:15-1:35	Show severity/evidence.
1:35-1:55	Show multimodal risk calculation.
1:55-2:15	Show GIS hotspot.
2:15-2:30	Expert confirms/corrects.
2:30-2:45	Show multilingual advisory + alert.
2:45-3:00	Run follow-up and close with hardware-ready architecture.

46. Future Physical Hardware Integration



Future work: field calibration, weather integration, real pest-trap imaging, broader crops, edge inference and larger geospatial datasets.

47. Project Deliverables Checklist

Deliverable	Required artifact
Architecture	System + service flow diagrams
SRS	Functional/non-functional requirements
AI	Training notebook/scripts + model + metrics
Dataset	Versioned metadata and split definition
Backend	API + database + tests
Frontend	Farmer + expert/officer interfaces
Simulator	Device UI + scenarios + replay
Risk	Feature definitions + scoring implementation
GIS	Zone map/hotspot layer
Knowledge base	Approved advisory records + versions
Testing	Test cases + results
Deployment	Docker/configuration + setup guide
Demo	Reproducible scenario and fallback
Pitch	Problem, architecture, impact, metrics, limitations

48. Final Definition of Done

```
SCOPE FROZEN
↓
AI BASELINE MEASURED
↓
END-TO-END SCAN WORKS
↓
SIMULATOR WORKS
↓
RISK WORKS
↓
GIS WORKS
↓
EXPERT WORKS
↓
ADVISORY WORKS
↓
ALERT/FOLLOW-UP WORKS
↓
TESTS PASS
↓
DEMO REPRODUCIBLE
↓
DOCUMENTATION COMPLETE
```

49. Engineering Rules

- Build the smallest complete vertical slice first.
- Do not add hardware merely for appearance; simulate it correctly and keep interfaces real.
- Do not invent AI accuracy; report measured metrics.
- Do not turn environmental correlation into causal certainty.
- Do not give unvalidated chemical instructions.
- Keep all simulated/cached data visibly labelled.
- Version datasets, models, rules and advisory content.
- Design every module so future physical-device integration is an adapter change, not a rewrite.

50. Final Product Definition

Crop Health Intelligence Platform: a software-first, hardware-ready decision-support system that detects supported crop diseases/pests from images, estimates severity, combines environmental and pest observations with farm history, forecasts rising risk, maps surveillance hotspots, routes uncertain cases to

experts, provides validated multilingual guidance, generates alerts and follow-up tasks, and stores outcomes for future improvement.

This document is the engineering master plan. Implementation should proceed phase-by-phase, with the MVP end-to-end workflow protected as the highest priority.